

Presentation 2

STAT 390 | Winter 2026 | Team 6

Northwestern

Data Pre-Processing (Programming)

Henry Chau

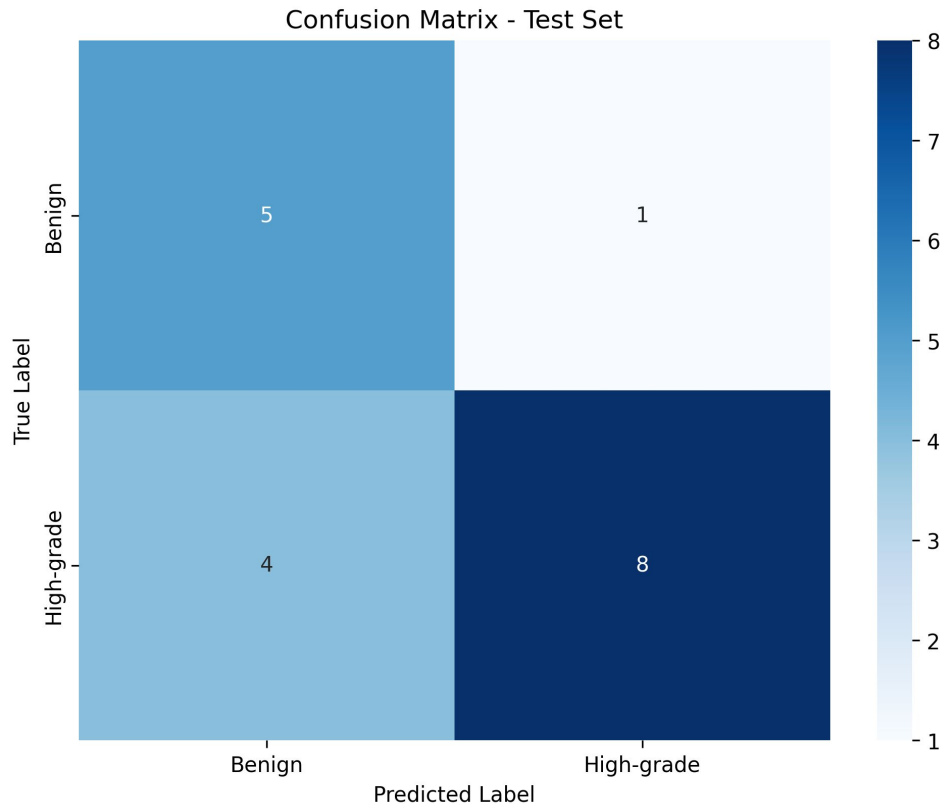
Mandatory tasks (1-4, 8) implementation

- Task 3 switched optimizer from Adam to AdamW based on team 5's suggestion for faster gradient descent convergence [↗](#)
- Task 4 removed saturation jittering to remove some excess noise introduced to the training data [↗](#)
- Task 8 added persistent workers reduce cpu idle between running epochs [↗](#)
- Task 1 implements the KimiaNet model which unfortunately didn't have much documentation to look at but we remapped the KimiaNet trained weights to DenseNet [↗](#)
- Task 2 changes the behavior of how the patches are transformed, I filtered out all patches smaller 32x32 and also attempted to remove cropping entirely but it was not possible because the model expects a certain size so it is now using `transform.RandomCrop()` [↗](#)

KimiaNet results

- Unfortunately, for whatever reason the model performed much worse than previously
- Test loss: .7272 >> .7648
- Test accuracy: .7778 >> .7222
- Now the model has the opposite problem as before, low recall for high-grade which is a problem

```
[trd1531@quser44 e32998]$ seff 8699426
Job ID: 8699426
Cluster: quest
User/Group: trd1531/trd1531
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 16
CPU Utilized: 1-15:04:47
CPU Efficiency: 62.88% of 2-14:08:48 core-walltime
Job Wall-clock time: 03:53:03
Memory Utilized: 24.99 GB
Memory Efficiency: 19.53% of 128.00 GB (128.00 GB/node)
```



Standardizing patches and investigating abnormalities

- As part of the data processing pipeline, I was trying to use regex to match all relevant information from the patch filename to identify distribution of slices in preparation for task 11
- I worked on updating some code in `data_utils.py` to accomplish my goals, one of which was to standardize the patch filename scheme. The previous code did not make attempts to coerce the filenames to the standard and the regex was lenient
- Using a stricter regex: `"case_[^0]\d*_\w+_\d+_(h&e|melan|sox10)_patch\d+.png"`, multiple cases of non-standard filenames were found but the vast majority were handled by coercing to the standard [!\[\]\(3da2b303d29c1ea489bbe26a3f5ac664_img.jpg\)](#)

Standardizing patches and investigating abnormalities

- The new regex code finds invalid images that the previous missed, included below are a whole slide annotated image that shouldn't be there, some duplicate images, and images with the same file name in case 92 patch 71&72 that are actually different
- There are so few relative to the total number of patches so it is not too important but I did exclude them from the model for data quality

```
69 Found and excluding 1 files not following naming convention:
70   case_056_match_1_h&e.png
71 Found and excluding 64 potentially duplicate files:
72   case_43_match_1_h&e_patch2 2.png
73   case_92_match_1_h&e_patch72 (1).png
74   case_43_match_1_h&e_patch30 2.png
75   case_43_match_1_h&e_patch3 2.png
76   case_43_match_1_melan_patch32 2.png
77   ... and 59 more
78 25878 out of 25943 non-standard file names were successfully coerced to standard.
79 Found 83530 patch files
80 These cases have at least one slide with more than 5 slides per stain
```

Task 11 overview

- I created a new function in data_utils.py, make_pseudocases() which attempts to identify specifically benign cases where at least one stain has more than 5 slices
- I only found 6 benign cases with excess slices, and they were split into pseudo cases by breaking each stain's slices roughly equally and assigning them to new cases
- Due to time constraints I did not come up with a more sophisticated method to split the slices to avoid potential data leakage. (The current code cannot detect data leakage)
- 95 cases are now available, up from 85

```
80 These cases have at least one stain with more than 5 slices per stain
81 |   melan   h&e  sox10
82 22    3.0   13.0   3.0
83 24    4.0   10.0   2.0
84 25    2.0    6.0   2.0
85 26    8.0   17.0   4.0
86 27    4.0    6.0   2.0
87 82    1.0    6.0   1.0
88 Loaded new 120 labels
89 Grouped into 393 slices
90 Mapped 381 slices to classes
91 Class distribution: {0: 143, 1: 238}
92
```

Task 11 implementation

- Benign to high-grade patch count ratio is slightly better, about 1/2 compared to 1/3 last time in the testing data but it could also just be the result of creating a new split
- Some potential issues to look out for are data leakage, new cases are currently able to be assigned to training, validation, or testing; now some cases have very few slices per stain, case 82 is problematic because one of its split out case has no melan or sox10 stains
- The code is still running so there are no final results yet

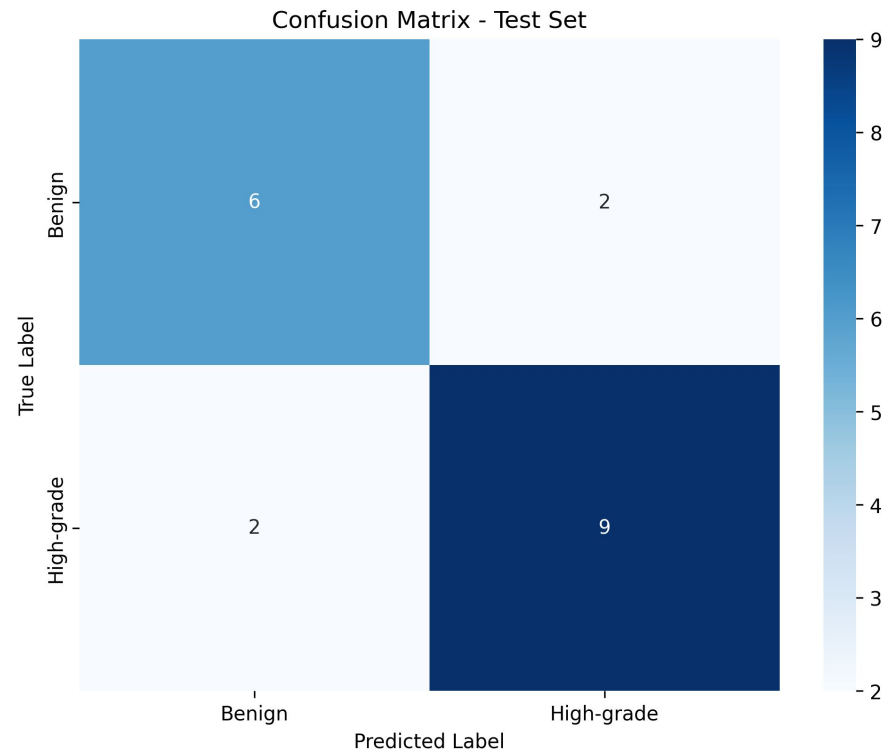
```
137 Total cases: 95
138 Total patches: 79,941
139 =====
140
141 Patch count by class for Train:
142   Benign (0):    16384 patches
143   High-grade (1): 33775 patches
144
145 Patch count by class for Validation:
146   Benign (0):    4737 patches
147   High-grade (1): 10488 patches
148
149 Patch count by class for Test:
150   Benign (0):    4560 patches
151   High-grade (1): 9997 patches
152
```


Task 11 results

- Benign to high-grade patch count ratio is slightly better, about $1/2$ compared to $1/3$ last time in the testing data but it could also just be the result of creating a new split
- Some potential issues to look out for are data leakage, new cases are currently able to be assigned to training, validation, or testing; now some cases have very few slices per stain, case 82 is problematic because one of its split out case has no melan or sox10 stains
- The code is still running so there are no final results yet

Task 11 implementation

- This is the result after data augmentation and Vicky's additions that was not ready in time for the recording



Experimental model improvement (Programming)

Vicky Xu

Moderate tasks (5,6) implementation

- Current Model: A Hierarchical Attention Multiple Instance Learning (MIL) framework.
 - Linearity Constraint: The previous attention mechanism was linear between -1 and 1, limiting the model's ability to learn complex, non-linear diagnostic patterns.
 - Peakiness Attention Scores: A few patches were dominating the case embedding, making the model prone to noise and poor generalization.
- Task 5: Add entropy regularization for attention scores to avoid peakiness attention scores.
- Task 6: Use Gated attention instead of the current one.

task5: Entropy regularization

trainer.py
function train_epoch

- To prevent "peaky" attention scores where a few patches dominate the case embedding, I add a penalty term based on the entropy of the attention distribution.
- Higher entropy means the attention weights are more evenly distributed, rather than sharply focused on a few patches.
- The penalty term is defined as $-\lambda H(w)$. During gradient descent, this term encourages a more balanced distribution by increasing smaller weights and decreasing larger ones.

```
# 1. Standard CrossEntropy Loss
ce_loss = self.criterion(outputs, label)

# 2. Entropy Regularization for each attention layer
case_entropy = AttentionPool.compute_entropy(weights_dict['case_weights'])

# Penalty = -lambda * H(w)
entropy_lambda = getattr(self, 'entropy_lambda', 0.0005)
total_loss = ce_loss - (entropy_lambda * case_entropy)
```

task5: Entropy regularization

trainer.py
function train_epoch

- Start with $\lambda = 0.001$ to aggressively counter peaky attention and force the model to integrate more spatial information from the slides.
- Stronger regularization helped stability but slightly suppressed the model's ability to highlight specific malignant regions.
- I set the coefficient to $\lambda = 0.0005$. This smaller value makes the entropy constraint “softer,” meaning it still encourages a balanced attention distribution but does not overly restrict it.

```
# 1. Standard CrossEntropy Loss
ce_loss = self.criterion(outputs, label)

# 2. Entropy Regularization for each attention layer
case_entropy = AttentionPool.compute_entropy(weights_dict['case_weights'])

# Penalty = -lambda * H(w)
entropy_lambda = getattr(self, 'entropy_lambda', 0.0005)
total_loss = ce_loss - (entropy_lambda * case_entropy)
```

task6: Gated attention

models.py
class AttentionPool

- The Content Path (V): Uses nn.Tanh() to extract the latent features of the patches.
- The Gate Path (U): Uses nn.Sigmoid() to act as a learnable "filter" or "gate".
- I compute $A_v \times A_u$, where the Sigmoid gate scales the Tanh output to control how much information passes through. This gating adds nonlinearity and helps the model capture subtle pathological patterns.

```
def __init__(self, input_dim: int, hidden_dim: int = 128, dropout: float = 0.0):  
    super().__init__()  
    # Gated Attention: V (tanh) and U (sigmoid)  
    self.attention_V = nn.Sequential(  
        nn.Linear(input_dim, hidden_dim),  
        nn.Tanh()  
    )  
    self.attention_U = nn.Sequential(  
        nn.Linear(input_dim, hidden_dim),  
        nn.Sigmoid()  
    )  
    self.attention_weights = nn.Linear(hidden_dim, 1)  
    self.dropout = nn.Dropout(dropout)
```

result

- Test Accuracy: Increased from 0.7222 to 0.7895.
- Test Loss: Decreased from 0.7648 to 0.5891.
- By rewarding higher entropy, it forced the model to "spread its focus" across more patches. This led to a more robust representation of each case, which is reflected in the significantly lower Test Loss (0.5891).
- The increased capacity to capture complex features allowed the model to better distinguish between subtle pathology classes, resulting in the Test Accuracy of 0.7895.

Test Results:
Test Loss: 0.7648
Test Accuracy: 0.7222



Job ID: 8828458
Start time: Mon Feb 16 10:11:34 CST 2026

Test Results:
Test Loss: 0.5891
Test Accuracy: 0.7895

result

- High-Confidence (Case 59, 1):
 - Probs 0.88; very decisive and correct.
 - Gated Attention: Learns complex non-linear diagnostic signals.
- Stable Decisions (Case 28, 2025):
 - Correct despite borderline probabilities
 - Entropy: Prevents "peaky" focus on noise; ensures broader tissue integration.
- High Precision (Case 19, 52)
 - Consistently high scores for true positives.
 - Dual-Path V & U: Effectively gates out non-diagnostic slide background.

```
case_id,true_label,predicted_label,prob_benign,prob_high_grade,correct
3026,0,0,0.7178985,0.2821015,True
2082,0,1,0.18831886,0.81168115,False
3022,0,1,0.3635498,0.63645023,False
28,0,0,0.5079258,0.49207422,True
2025,0,0,0.5109604,0.4890396,True
93,0,0,0.5538344,0.44616562,True
85,0,0,0.60953796,0.39046207,True
45,0,0,0.5829549,0.41704512,True
19,1,1,0.17044246,0.82955754,True
52,1,1,0.262667,0.737333,True
55,1,1,0.4094148,0.59058523,True
3,1,1,0.2887337,0.71126634,True
51,1,1,0.37775657,0.6222434,True
50,1,0,0.60583365,0.3941663,False
59,1,1,0.116557196,0.8834429,True
18,1,1,0.3780456,0.6219544,True
1,1,1,0.114462785,0.8855372,True
7,1,0,0.70034736,0.29965267,False
88,1,1,0.40287113,0.5971288,True
```

limitation

1. High Computational Cost

Training currently runs on CPUs, taking ~15 minutes per epoch. This limits extensive hyperparameter tuning (e.g., broader λ search) and efficient 5-fold cross-validation.

-> Use GPU instead??

2. Risk of Over-Smoothing from Entropy Regularization

While entropy regularization reduces peaky attention, excessive λ may overly diffuse attention and miss small malignant clusters.

Misclassified cases (e.g., Case 50, Case 7) suggest that some samples may require more localized focus than the current global constraint allows.

```
case_id,true_label,predicted_label,prob_benign,prob_high_grade,correct
3026,0,0,0.7178985,0.2821015,True
2082,0,1,0.18831886,0.81168115,False
3022,0,1,0.3635498,0.63645023,False
28,0,0,0.5079258,0.49207422,True
2025,0,0,0.5109604,0.4890396,True
93,0,0,0.5538344,0.44616562,True
85,0,0,0.60953796,0.39046207,True
45,0,0,0.5829549,0.41704512,True
19,1,1,0.17044246,0.82955754,True
52,1,1,0.262667,0.737333,True
55,1,1,0.4094148,0.59058523,True
3,1,1,0.2887337,0.71126634,True
51,1,1,0.37775657,0.6222434,True
50,1,0,0.60583365,0.3941663,False
59,1,1,0.116557196,0.8834429,True
18,1,1,0.3780456,0.6219544,True
1,1,1,0.114462785,0.8855372,True
7,1,0,0.70034736,0.29965267,False
88,1,1,0.40287113,0.5971288,True
```